C# 텀프로젝트

모터 제어 및 실시간 데이터 모니터링 툴 개발

202373081 황유

프로젝트 개요

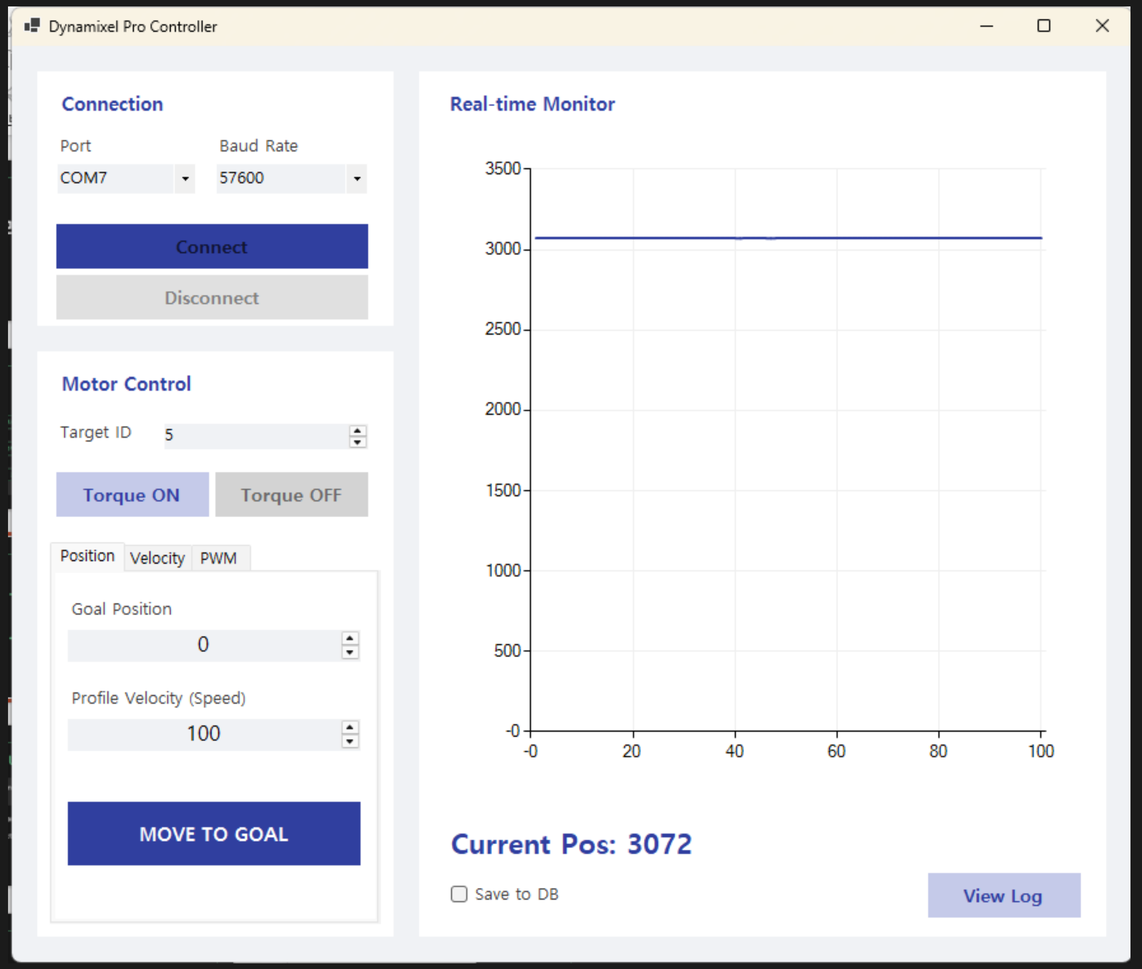
프로젝트 배경	상용 툴을 사용하지 않고 모터를 직접 제어하는 소프트웨어 개발
프로젝트 제작 기간	2025. 12. 01 ~ 2025. 12. 16
개발 환경	• HW: U2D2, Dynamixel MX-64 • SW: VS 2022, C# (.NET 8.0 Windows Forms) • 통신: Protocol 2.0, Serial (UART) • DB: SQLite

02

UI Design

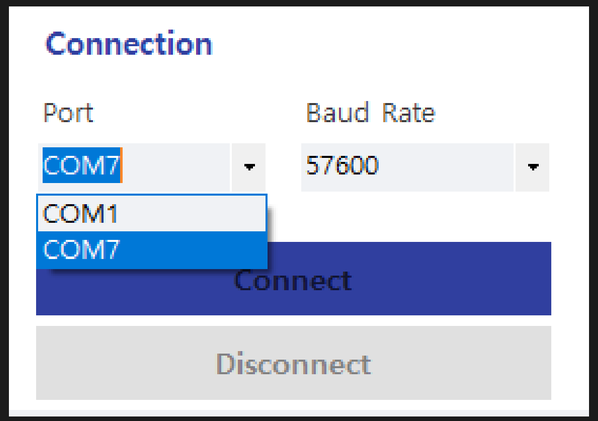
통신 연결 →

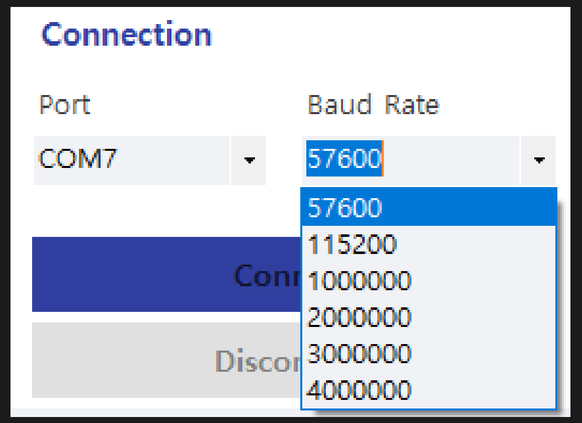
모터 제어 →

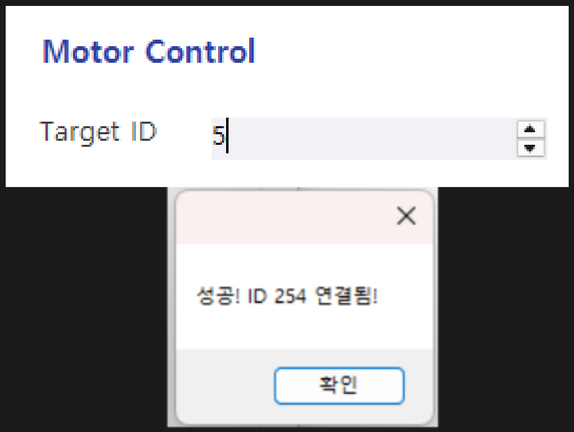


← 실시간 모니터링

모터 통신 기능







1. COM 포트 설정

2. Baudrate 설정

3. 모터 ID 스캔

모터 통신 기능

```
serialPort.Open();
serialPort.RtsEnable = true;
serialPort.DtrEnable = true;
```

하드웨어 통신 안정성을 위해
RTS/DTR 신호 활성화

```
if (DoPing(targetId))
{
    btnConnect.Enabled = false;
    btnDisconnect.Enabled = true;
    monitorTimer.Start();
    MessageBox.Show($"성공! ID {targetId} 연결됨!");
}
```

1. COM 포트 설정
2. Baudrate 설정

3. 모터 ID 스캔

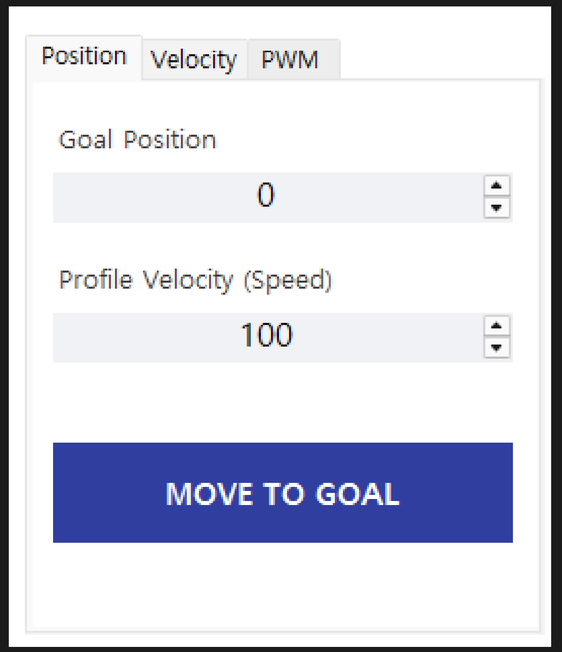
통신 연결에서의 예외 처리

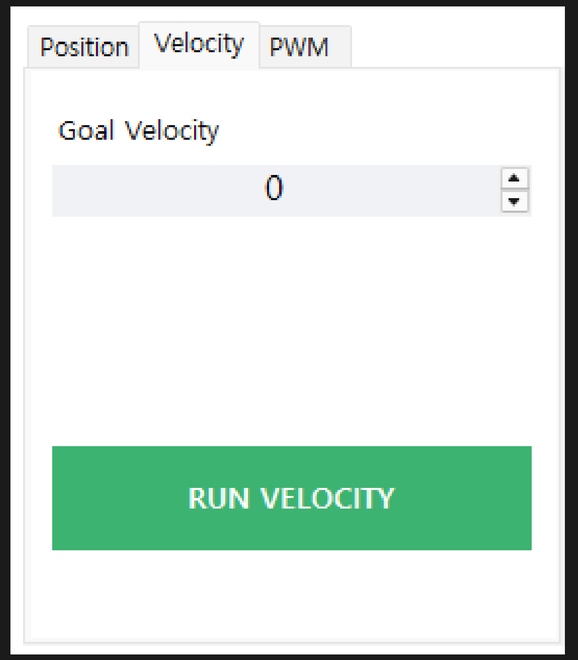
```
private void InitDatabase()
{
    try
        { 중략 }
    catch (Exception ex) { 중략 }
    }
}
```

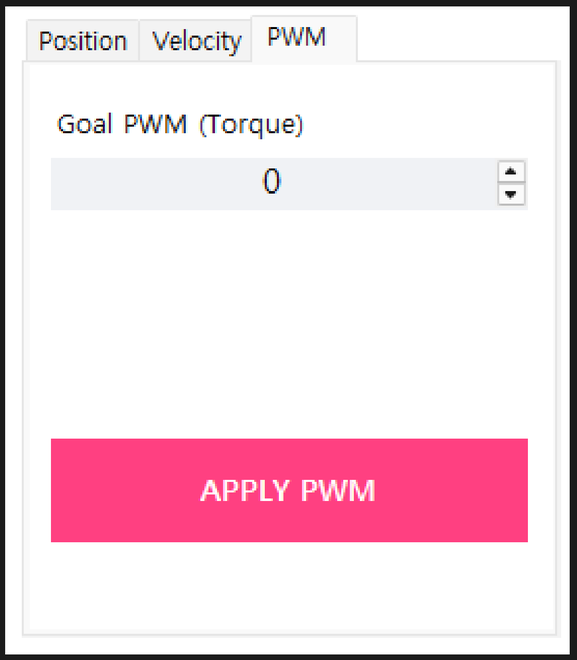
외부 장치(Hardware)나 파일(DB)을 다룰 때 발생할 수 있는 런타임 오류를 try-catch 문으로 처리하여 프로그램이 비정상 종료되지 않도록 함

모터 제어 기능

> 3가지 기능을 탭 구조의 UI로 설계







1. 위치 제어
2. 속도 제어
3. 전류 제어

모터 제어 기능

실제 다이나믹셀 패킷 구조 >

Header				ID	Length		Inst	Param				CRC	
FF	FF	FD	00	17	07	00	02	40	00	04	00	4D	25

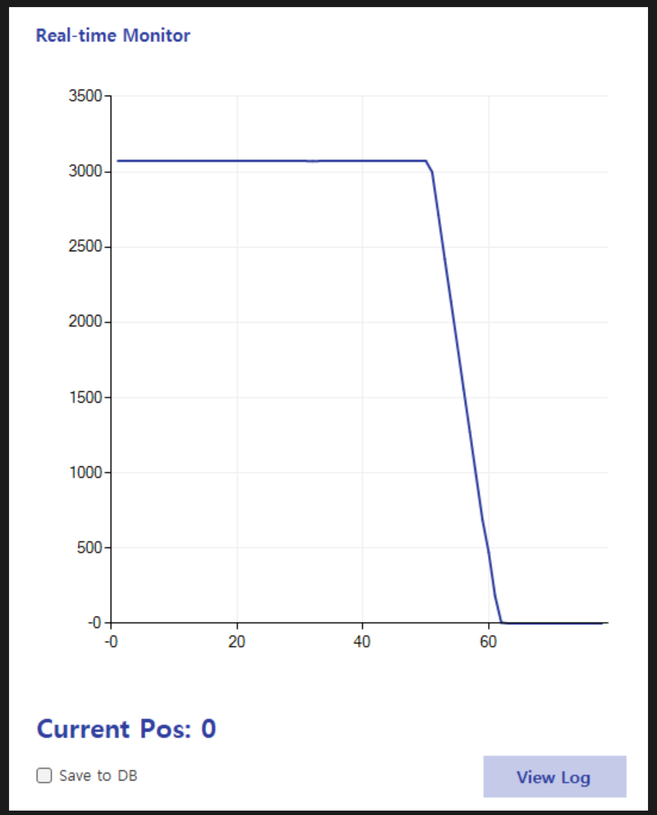
코드에서 구현한 패킷 구조 >

```
byte[] movePacket = new byte[] {
0xFF, 0xFF, 0xFD, 0x00,
0x17,
0x09, 0x00,
0x03,
0x74, 0x00,
0x00, 0x00, 0x00, 0x00,
0xCB, 0x48
};
```

SDK에 의존하지 않고, 바이트 단위로 패킷 직접 구성

03

모니터링 기능



ex) Goal Position 제어를 통해
0으로 위치 변경

Timer를 이용한 주기적 데이터 Poling, 그래프 시각화

모니터링 기능

MonitorTimer_Tick 함수 >

```
if (serialPort.BytesToRead >= 15)
{
    byte[] buffer = new byte[serialPort.BytesToRead];
    serialPort.Read(buffer, 0, buffer.Length);
    int presentPos = ParsePosition(buffer);
    if (presentPos != -1) UpdateUI(presentPos);
}
```

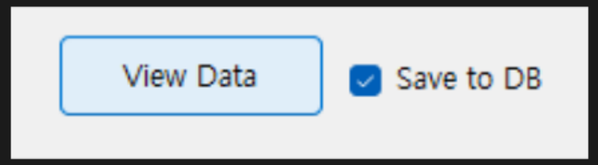
함수를 통해 수신 데이터 파싱

모니터링 기능

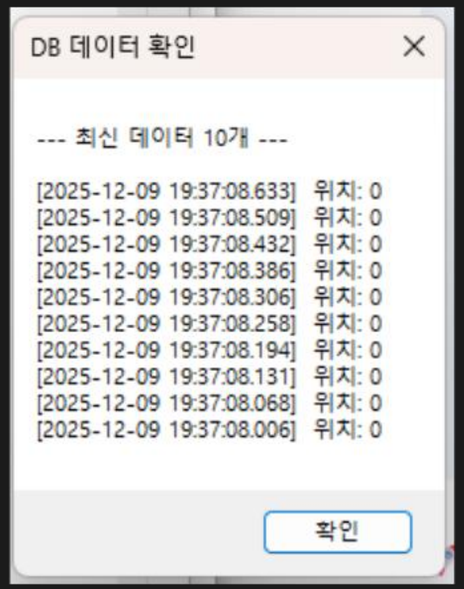
```
monitorTimer = new System.Windows.Forms.Timer();
monitorTimer.Interval = 50;
monitorTimer.Tick += MonitorTimer_Tick;
```

이벤트 핸들러와 델리게이트를 사용하여 타이머가 틱을 발생시킬 때마다 특정 메서드가 실행되도록 구현

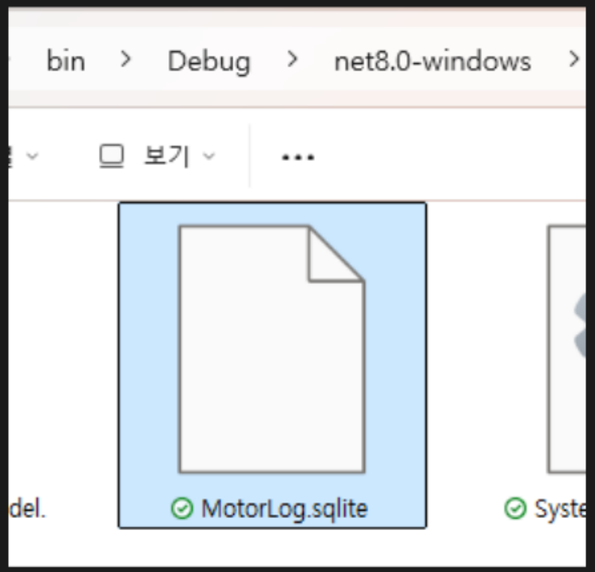
데이터베이스 로깅



체크박스 및 팝업창



데이터 확인



로컬 데이터 저장

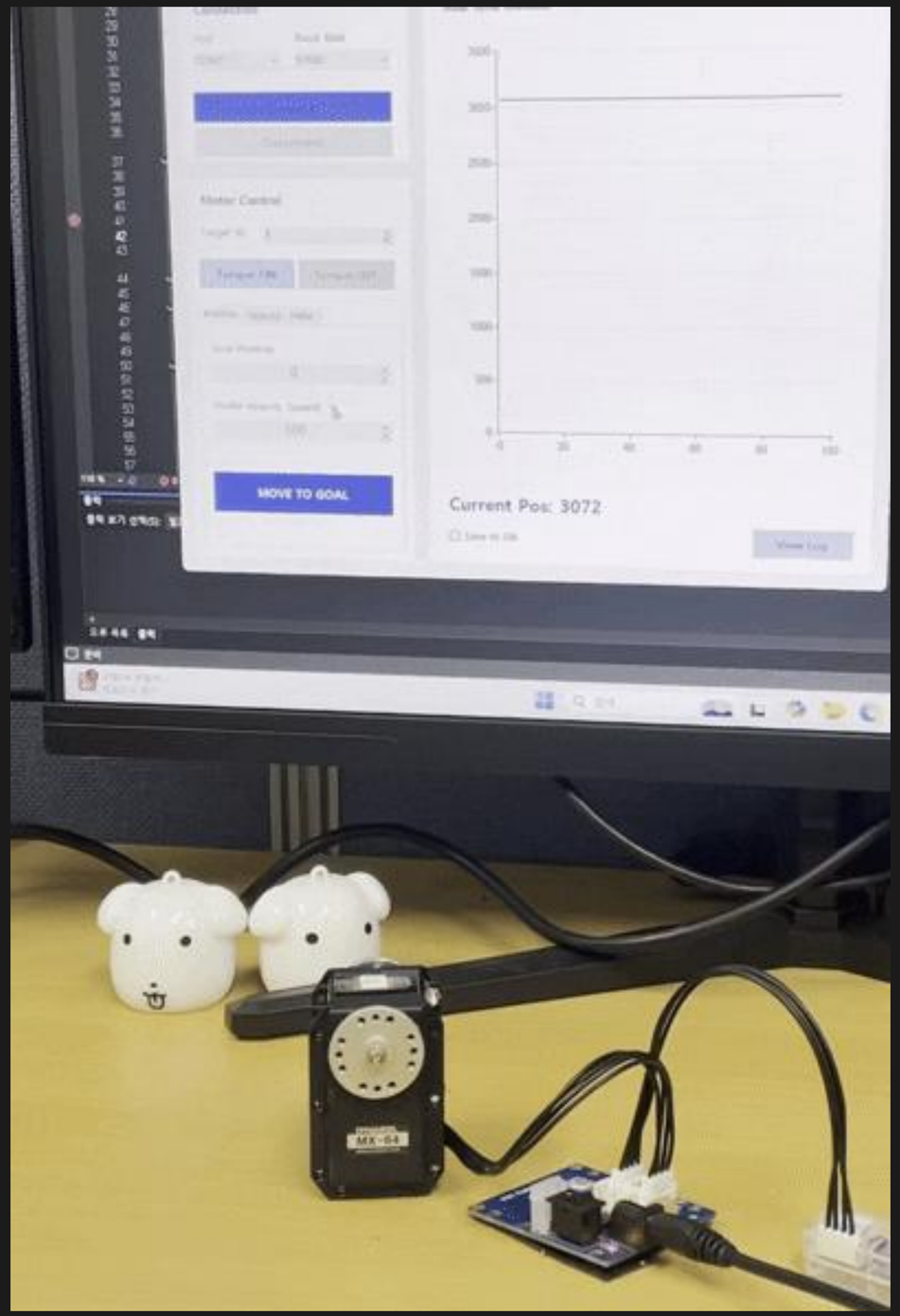
SQLite를 이용한 로컬 데이터 저장 및 조회

데이터베이스 로깅

```
if ((DateTime.Now - lastSaveTime).TotalSeconds >= 1.0)
{
    try
    {
        string sql = "INSERT INTO MotorData (Timestamp, Position) VALUES (@time, @pos)";
        SQLiteCommand cmd = new SQLiteCommand(sql, dbConn);
        cmd.Parameters.AddWithValue("@time", DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss.fff"));
        cmd.Parameters.AddWithValue("@pos", pos);
        cmd.ExecuteNonQuery();

        lastSaveTime = DateTime.Now;
    }
```

샘플링 레이트(1Hz)를 별도로 구현하여 시스템 효율을 높임


MOVE TO GOAL
Current Pos: 3072
MX-64

프로젝트 요약 및 성과

문제 해결 과정	C#의 SerialPort 설정에서 RTS/DTR 제어 신호를 강제로 활성화하여 송수신 회로가 정상적으로 동작하도록 설계
핵심 역량	상용 SDK 없이 프로토콜 2.0 패킷 생성 및 파싱 로직 직접 구현, 임베디드 통신 기술 확보
완성도	제어 - 모니터링 - 데이터 저장으로 이루어지는 제어 시스템 구현

별첨 - 데이터 수신 및 파싱 로직

```
private int ParsePosition(byte[] buffer)
{
    for (int i = 0; i < buffer.Length - 10; i++)
    {
        if (buffer[i] == 0xFF && buffer[i + 1] == 0xFF && buffer[i + 2] == 0xFD && buffer[i + 3] == 0x00)
        {
            if (buffer[i + 7] == 0x55)
            {
                if (buffer[i + 8] != 0) return -1;
                int pos = BitConverter.ToInt32(buffer, i + 9);
                return pos;
            }
        }
    }
    return -1;
}
```

- 들어오는 데이터 스트림에서 헤더(Header)를 찾고, 실제 데이터(위치값)를 추출해내는 알고리즘
  - ParsePosition이라는 메서드로 독립시킴

별첨 - CRC 계산 로직

```
private static readonly ushort[] crc_table = {
    0x0000, 0x8005, 0x800F, 0x000A, 0x811B, 0x011E, 0x0114, 0x8111,
    0x0233, 0x8236, 0x823C, 0x0239, 0x8328, 0x032D, 0x0327, 0x8322,
    0x0466, 0x8463, 0x8469, 0x046C, 0x857D, 0x0578, 0x0572, 0x8577,
    0x0655, 0x8650, 0x865A, 0x065F, 0x874E, 0x074B, 0x0741, 0x8744,
    0x08CC, 0x88C9, 0x88C3, 0x08C6, 0x89D7, 0x09D2, 0x09D8, 0x89DD,
    0x0AFF, 0x8AF5, 0x8AFD, 0x0AF5, 0x8BED, 0x0BE8, 0x0BE2, 0x8BE7,
```

< CRC Table (중략)

```
private ushort UpdateCRC(ushort crc_accum, byte[] data_blk_ptr, int data_blk_size)
{
    for (int j = 0; j < data_blk_size; j++)
    {
        int i = ((int)(crc_accum >> 8) ^ data_blk_ptr[j]) & 0xFF;
        crc_accum = (ushort)((crc_accum << 8) ^ crc_table[i]);
    }
    return crc_accum;
}
```

통신 무결성을 검증하는 알고리즘 구현
- 사전에 정의된 CRC Table을 사용하여 계산 속도 최적화
- 현재 CRC 값과 입력 데이터 간의 XOR 연산 및 테이블 참조